Predictive Serverless Cold-Start Optimizer

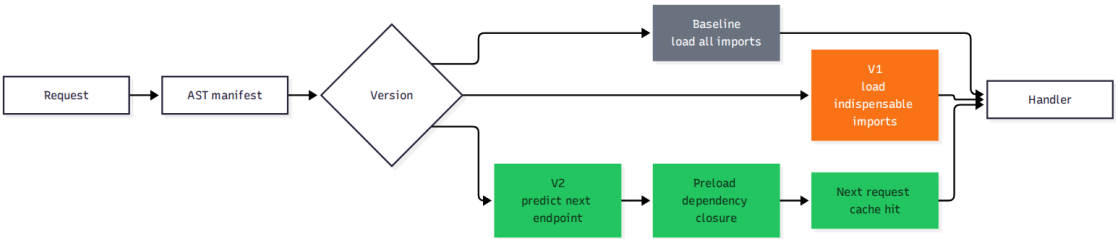
From static import deferral to predictive preloading

Rahul Saravanan (231IT054) Shishir Hegde (231IT067)

Department of Information Technology
National Institute of Technology Karnataka, Surathkal

October 2026

The problem is not only the handler



- A fresh environment imports code before it can serve the first request.
- Optional SDKs can dominate that path even when a request never uses them.
- Deferring imports can move heavy work into the handler.

Question

Can we reduce startup loading without moving the same cost into user-visible latency?

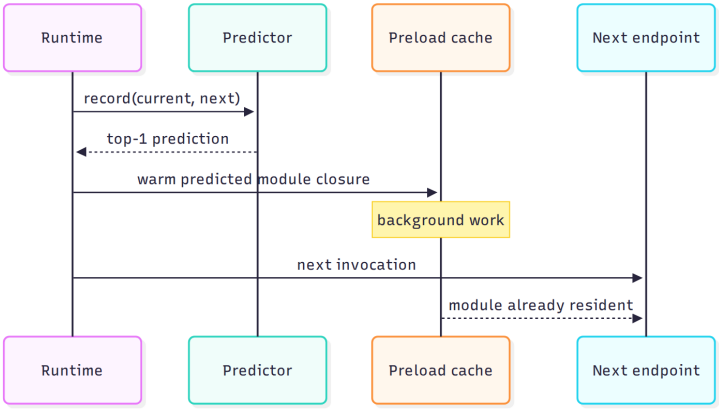
From the true baseline to PCSO

Layer	Baseline	V1 (FaaSLight)	PCSO
Startup loading	Eager full dependency set	Indispensable imports only	Indispensable imports only
Runtime knowledge	None	AST dependency manifest	AST manifest plus next-hop model
Heavy dependencies	Paid on request path	Deferred to handler	Preloaded before predicted request
Working set	Rebuilt each cold start	No predictive cache	Predicted module closure stays warm

The comparison used in this presentation

The eager runtime is the true baseline. V1 is the FaaSLight minimum-improvement reference; PCSO (Predictive Cold Start Optimizer) is the final system. Preload time is tracked separately from request latency.

How PCSO removes the shifted-latency cost



- Static analysis says what can be deferred.
- The transition model says what is likely to be needed next.
- The preloader pays that cost before the next request arrives.

PCSO prediction under the hood

Online first-order Markov model

For every observed transition $A \to B$, PCSO increments the count in row A, column B.

$$P(B \mid A) = \frac{\text{count}(A \to B)}{\sum_x \text{count}(A \to x)}$$

The highest-probability next endpoint is selected when its confidence is at least 0.35.

Small example

If $A \to B$ occurs 7 times and $A \to C$ occurs 3 times, PCSO predicts B with probability 0.70.

What happens next

1. Observe the current endpoint and record the completed transition.
2. Predict the next endpoint from the updated transition row.
3. Import the predicted endpoint and its local dependency closure into the warm set.
4. On the next request, reuse the resident module instead of importing it on the critical path.

Miss behavior

If confidence is low or the request differs from the prediction, PCSO falls back to the V1 static-loading path; correctness is unchanged.

Benchmark design

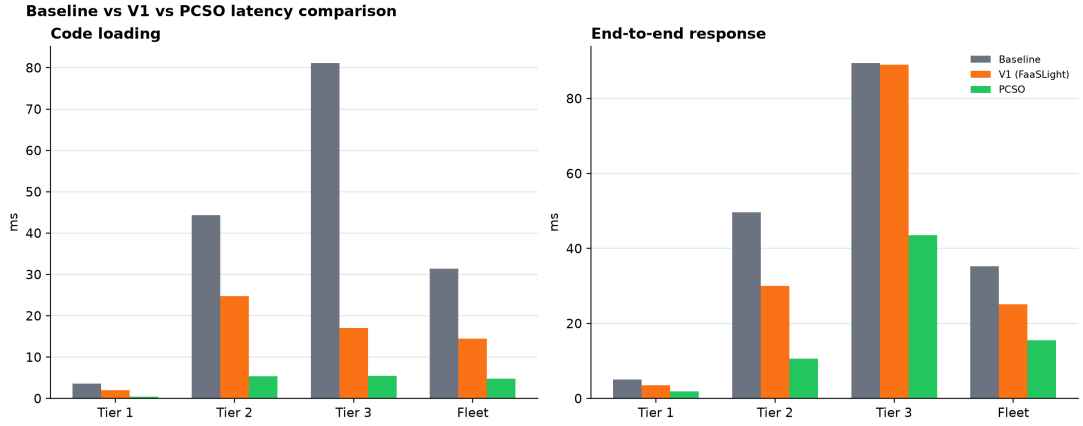
Workloads

- Tier 1: lightweight CRUD and utilities
- Tier 2: medium enterprise services
- Tier 3: OCR, fraud, and media pipelines
- Blended fleet: 50 / 35 / 15%

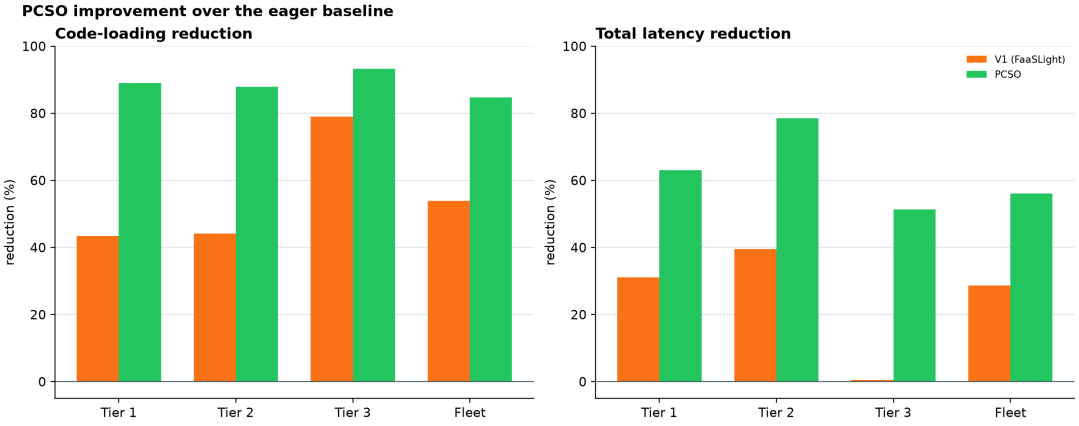
Protocol

- Fixed random seed
- 24 invocations per tier; 32 blended
- Isolated cold-start samples
- Mean, p95, memory, and cache hit rate

Latency: Baseline vs V1 vs PCSO



Improvement over the true baseline



Measured results

	Code loading (ms)			Total (ms)		
	Baseline	V1	PCSO	Baseline	V1	PCSO
Tier 1	3.62	2.05	0.40	5.08	3.50	1.88
Tier 2	44.32	24.76	5.37	49.61	30.02	10.63
Tier 3	81.15	17.05	5.46	89.47	88.98	43.52
Fleet	31.36	14.45	4.77	35.29	25.16	15.48

Fleet result

V1: 53.9% less code-loading time and 28.7% less total latency.
PCSO: 84.8% less code-loading time and 56.1% less total latency.

Why PCSO improves on the true baseline

1. V1 / FaaSLight minimum improvement

Static AST analysis removes optional model, OCR, and media modules from startup. V1 reduces blended code-loading time by 53.9%.

2. PCSO: prediction + preloading

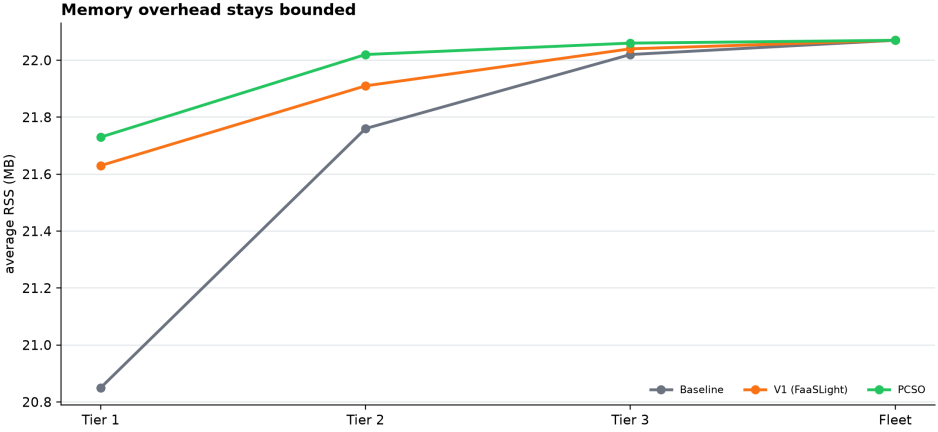
PCSO adds endpoint-transition learning and warms the predicted dependency closure, avoiding the deferred-import penalty inside the handler.

Together, these changes reduce the blended fleet from 35.29 ms to 15.48 ms per request; PCSO improves total latency by 56.1%.

Why Tier 3 improves sharply from V1 to PCSO

V1 defers the expensive OCR, ML, and media imports, but Tier 3 still pays for them inside the handler: 88.98 ms versus 89.47 ms for the baseline. PCSO predicts the next endpoint and preloads those heavy modules before the request, reducing Tier 3 latency to 43.52 ms (51.4% below baseline).

PCSO keeps memory approximately flat



Conclusion

- PCSO replaces eager full-graph loading with an AST-guided startup manifest.
- PCSO predicts the next endpoint and preloads its dependency closure.
- The final comparison is direct: Baseline vs PCSO.
- On the measured blended fleet, PCSO cuts total response latency by 56.1%.

Thank you

Questions